Quiero que me ayudes a generar la lógica de negocio real del backend de mi proyecto PBIO.

En este prompt quiero trabajar SOLO con la lógica del negocio.

No quiero hacer todavía las rutas REST.

Tampoco quiero Android ni la web.

La idea es esta:

REST
 ->
LÓGICA DE NEGOCIO
 ->
BBDD

La lógica de negocio tiene que poder funcionar sin saber si la petición viene de Android, de la web o de otro sitio.

Por eso no quiero que aquí utilices:

$_GET
$_POST
php://input

ni códigos HTTP.

Eso lo haremos después en el servidor REST.

DATOS

Tenemos conceptualmente:

```
Dispositivo = (
   id_dispositivo: N,
   uuid: Text,
   nombre: Text
)
```

y:

```
Medicion = (
   id_medicion: N,
   id_dispositivo: N,
```

uuid: Text,
tipo: Text,
valor: Z,
contador: N,
rssi: Z,
fecha_hora: Text
)

El RSSI puede ser opcional.

Los tipos de medición son:

CO2
TEMPERATURA
RUIDO

OBTENER O REGISTRAR DISPOSITIVO

Quiero esta operación:

uuid: Text, nombre: Text
-->
obtenerORegistrarDispositivo()
-->
Dispositivo

Tiene que buscar el dispositivo mediante su UUID.

Si existe:

lo devuelve.

Si no existe:

lo crea y lo devuelve.

No quiero que se puedan crear dos dispositivos con el mismo UUID.

REGISTRAR MEDICIÓN

Quiero:

medicion: Medicion
-->
registrarMedicion()
-->
Medicion

Cuando llamemos a esta operación recibiremos:

uuid
nombre_dispositivo
tipo
valor
contador
rssi

No recibiremos:

id_medicion
id_dispositivo
fecha_hora

La función tiene que:

1. validar los datos;
2. buscar el dispositivo por UUID;
3. crearlo si todavía no existe;
4. obtener su id_dispositivo;
5. guardar la medición;
6. dejar que la BBDD asigne fecha_hora;
7. devolver la medición registrada.

Las reglas son:

uuid obligatorio.

tipo obligatorio.

tipo solamente puede ser:

CO2
TEMPERATURA
RUIDO

valor tiene que ser entero y puede ser negativo.

contador tiene que ser entero entre 0 y 255.

rssi puede faltar y si existe puede ser negativo.

OBTENER MEDICIONES

Quiero:

obtenerMediciones()
-->
[Medicion]

Tiene que devolver todas las mediciones ordenadas desde la más reciente hasta la más antigua.

ÚLTIMA MEDICIÓN

Quiero:

tipo: Text
-->
obtenerUltimaMedicion()
-->
Medicion

Le pasamos por ejemplo:

CO2

y nos devuelve la última medición de CO2.

Lo mismo para:

TEMPERATURA
RUIDO

Si no existe ninguna, quiero poder distinguir ese caso claramente.

No pongas aquí un HTTP 404 porque eso pertenece al REST.

MEDICIONES DE UN DISPOSITIVO

Quiero:

uuid_dispositivo: Text
-->
obtenerMedicionesDispositivo()
-->
[Medicion]

Tiene que buscar el dispositivo mediante UUID y devolver sus mediciones desde la más nueva.

Quiero poder distinguir:

- dispositivo que no existe;
- dispositivo que existe pero todavía no tiene mediciones.

OBTENER DISPOSITIVOS

Quiero:

obtenerDispositivos()
-->
[Dispositivo]

Tiene que devolver los dispositivos registrados.

BASE DE DATOS

La base de datos ya tiene las tablas:

DISPOSITIVO
MEDICION

Utiliza PDO.

Quiero consultas preparadas.

No concatenes directamente valores externos dentro del SQL.

Intenta separar:

- conexión a BBDD;
- acceso a datos;
- lógica del negocio.

No quiero una función enorme que haga todo.

ERRORES

Quiero que la lógica pueda distinguir errores como:

datos incorrectos;

tipo no válido;

contador fuera de rango;

dispositivo inexistente;

medición inexistente;

error de base de datos.

Puedes usar excepciones o una solución sencilla equivalente.

Pero NO quiero utilizar aquí:

400
404
500

Eso lo hará después el servidor REST.

COMENTARIOS

Quiero comentarios en español.

No pongas comentarios obvios para cada línea.

Las funciones importantes tienen que tener una cabecera con nuestra notación.

Por ejemplo:

```
// ------------------------------------------------
// medicion: Medicion --> registrarMedicion() --> Medicion
//
// Valida y almacena una medición.
// ------------------------------------------------
```

Otro:

```
// tipo: Text --> obtenerUltimaMedicion() --> Medicion
```

Otro:

```
// uuid_dispositivo: Text
//     -->
// obtenerMedicionesDispositivo()
//     -->
// [Medicion]
```

Usamos:

N = naturales
Z = enteros
R = reales
B = booleanos
Text = texto

y:

[Tipo] = lista

TESTS

Quiero tests automáticos para esta lógica.

Quiero probar como mínimo:

1. Crear un dispositivo nuevo.

2. Volver a registrar el mismo UUID y comprobar que no lo duplica.

3. Registrar una medición correcta.

4. Registrar varias mediciones del mismo dispositivo.

5. Rechazar una medición sin UUID.

6. Rechazar una medición sin tipo.

7. Rechazar un tipo que no sea CO2, TEMPERATURA o RUIDO.

8. Permitir valor negativo.

9. Permitir RSSI negativo.

10. Permitir que no haya RSSI.

11. contador = 0.

12. contador = 255.

13. rechazar contador = -1.

14. rechazar contador = 256.

15. comprobar que obtenerMediciones() devuelve primero las nuevas.

16. comprobar obtenerUltimaMedicion().

17. comprobar obtenerMedicionesDispositivo().

18. comprobar que se diferencia dispositivo inexistente de dispositivo sin mediciones.

19. comprobar obtenerDispositivos().

Quiero que estos tests llamen directamente a la lógica de negocio.

No quiero hacer peticiones HTTP para probar esta parte porque todavía no estamos probando el servidor REST.

ANTES DE PROGRAMAR

Antes de modificar nada:

1. Mira el código existente.
2. Dime qué podemos reutilizar.

3. Mira si ya existe acceso a base de datos.
4. Dime qué archivos vas a crear o modificar.
5. Comprueba que no estás metiendo código REST en esta parte.

Después implementa la lógica y los tests.

Si puedes ejecutar los tests, ejecútalos.

Al final dime:

- qué archivos has creado;
- qué archivos has modificado;
- qué operaciones de negocio funcionan;
- qué reglas has implementado;
- qué tests has creado;
- cuáles pasan;
- cuáles fallan;
- y si has encontrado algún problema.

No quiero que en este prompt hagas REST, Android, navegador, UX, usuarios ni login.

Quiero solamente la lógica de negocio real del backend.